

DSA Patterns

C# Interview Reference · 12 Patterns · 80+ Problems

Templates · Complexity · Pitfalls · Practice Problems

Array

4 patterns

Linear

2 patterns

Graph

3 patterns

Tree

2 patterns

DP

1 pattern

CONTENTS

01	Two Pointers	Time $O(n)$ · Space $O(1)$
02	Sliding Window	Time $O(n)$ · Space $O(k)$ or $O(1)$
03	Hashing	Time $O(n)$ · Space $O(n)$
04	Binary Search	Time $O(\log n)$ · Space $O(1)$
05	Stack	Time $O(n)$ · Space $O(n)$
06	Queue / BFS	Time $O(V + E)$ · Space $O(V)$
07	DFS / Backtracking	Time $O(b^d)$ backtrack · Space $O(V)$ stack
08	Linked List	Time $O(n)$ · Space $O(1)$
09	Tree / Recursion	Time $O(n)$ · Space $O(h)$
10	Heap / Priority Queue	Time $O(n \log k)$ · Space $O(k)$
11	Dynamic Programming	Time $O(n^2)$ typical · Space $O(n)$ or $O(n^2)$
12	Graph Algorithms	Time $O((V+E) \log V)$ · Space $O(V + E)$



Two Pointers

Array

T: $O(n)$

S: $O(1)$

WHEN TO USE

- Sorted array
- Pair / triplet sum
- Max area / container
- Palindrome check
- Remove duplicates

CODE TEMPLATES

Opposite directions

```
int left = 0, right = arr.Length - 1;
while (left < right)
{
    if (arr[left] + arr[right] == target)
        return new[] { left, right };
    else if (arr[left] + arr[right] < target)
        left++;
    else
        right--;
}
```

Same direction (remove duplicates)

```
int slow = 0;
for (int fast = 0; fast < arr.Length; fast++)
{
    if (arr[fast] != val)
        arr[slow++] = arr[fast];
}
```

3-pointer (Dutch flag partition)

```
int lo = 0, mid = 0, hi = arr.Length - 1;
while (mid <= hi)
{
    if (arr[mid] == 0) Swap(arr, lo++, mid++);
    else if (arr[mid] == 1) mid++;
    else Swap(arr, mid, hi--);
}
```

PRACTICE PROBLEMS

easy	Two Sum II (sorted)
easy	Valid Palindrome
med	Container With Most Water
med	3Sum
easy	Remove Duplicates from Sorted Array
hard	Trapping Rain Water
med	Sort Colors (Dutch Flag)

PITFALLS & TIPS

- Forget left < right guard → index crossing bug.
- Using two pointers on unsorted data — sort first.
- Triplet problems: fix one index, run two-pointer on remainder.



Sliding Window

Array

T: $O(n)$

S: $O(k)$ or $O(1)$

WHEN TO USE

- Fixed-size subarray
- Substring / subarray
- Variable-size window
- Longest / minimum range

CODE TEMPLATES

Fixed window (size k)

```
int windowSum = 0;
for (int i = 0; i < k; i++) windowSum += arr[i];
int maxSum = windowSum;

for (int i = k; i < arr.Length; i++)
{
    windowSum += arr[i] - arr[i - k];
    maxSum = Math.Max(maxSum, windowSum);
}
```

Variable window — expand right, shrink left

```
int left = 0, maxLen = 0;
var freq = new Dictionary<char, int>();

for (int right = 0; right < s.Length; right++)
{
    freq[s[right]] = freq.GetValueOrDefault(s[right]) + 1;

    while (freq.Count > k)
    {
        freq[s[left]]--;
        if (freq[s[left]] == 0) freq.Remove(s[left]);
        left++;
    }
    maxLen = Math.Max(maxLen, right - left + 1);
}
```

At-most-K trick (exactly $K = \text{atMost}(K) - \text{atMost}(K-1)$)

```
int AtMostK(int[] arr, int k)
{
    int left = 0, count = 0, result = 0;
    for (int right = 0; right < arr.Length; right++)
    {
        if (arr[right] == 1) count++;
        while (count > k) if (arr[left++] == 1) count--;
        result += right - left + 1;
    }
    return result;
}
// Answer = AtMostK(k) - AtMostK(k - 1)
```

PRACTICE PROBLEMS

easy	Max Average Subarray I
med	Longest Substring Without Repeating Chars
med	Longest Repeating Character Replacement
hard	Minimum Window Substring
hard	Sliding Window Maximum (Deque)
med	Fruit Into Baskets
hard	Subarrays with K Different Integers

PITFALLS & TIPS

Update answer BEFORE shrinking for 'longest' problems.

Shrinking left past right — guard with $\text{left} \leq \text{right}$.

Sliding Window Maximum uses a Monotonic Deque, not a heap.

'Exactly K' problems: $\text{atMost}(K) - \text{atMost}(K-1)$ trick.



Hashing

Array

T: $O(n)$

S: $O(n)$

WHEN TO USE

- Frequency count
- Duplicate detection
- $O(1)$ lookup
- Prefix sum + map

CODE TEMPLATES

Frequency map (Dictionary)

```
var freq = new Dictionary<int, int>();
foreach (var x in arr)
    freq[x] = freq.GetValueOrDefault(x) + 1;

// top-K using frequency
var topK = freq.OrderByDescending(kv => kv.Value)
    .Take(k).Select(kv => kv.Key).ToArray();
```

HashSet — existence / duplicate check

```
var seen = new HashSet<int>();
foreach (var x in arr)
{
    if (!seen.Add(x)) // Add() returns false on duplicate
        return true; // found duplicate
}
```

Prefix sum + HashMap (subarray sum = k)

```
var prefixCount = new Dictionary<int, int> { [0] = 1 };
int sum = 0, count = 0;

foreach (var x in arr)
{
    sum += x;
    count += prefixCount.GetValueOrDefault(sum - k);
    prefixCount[sum] = prefixCount.GetValueOrDefault(sum) + 1;
}
```

PRACTICE PROBLEMS

- | | |
|------|-------------------------------|
| easy | Two Sum |
| easy | First Non-Repeating Character |
| med | Group Anagrams |
| med | Subarray Sum Equals K |
| med | Longest Consecutive Sequence |
| med | Top K Frequent Elements |
| med | Find All Duplicates |

PITFALLS & TIPS

- Use GetValueOrDefault() — ContainsKey + indexer is two lookups.
- Forget prefixCount[0]=1 → misses subarrays starting at index 0.
- int[26] beats Dictionary when only lowercase letters involved.



Binary Search

Array

T: $O(\log n)$

S: $O(1)$

WHEN TO USE

- Sorted / rotated array
- Find exact element
- Minimize / maximize answer
- Monotonic search space

CODE TEMPLATES

Classic binary search

```
int lo = 0, hi = arr.Length - 1;
while (lo <= hi)
{
    int mid = lo + (hi - lo) / 2; // no overflow
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;
```

Lower bound — first index $\geq$ target

```
int lo = 0, hi = arr.Length; // open right
while (lo < hi)
{
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid;
}
return lo;
```

Binary search on answer (feasibility)

```
int lo = minPossible, hi = maxPossible, ans = -1;
while (lo <= hi)
{
    int mid = lo + (hi - lo) / 2;
    if (IsFeasible(mid)) { ans = mid; hi = mid - 1; }
    else lo = mid + 1;
}
return ans;
```

Rotated sorted array search

```

int lo = 0, hi = arr.Length - 1;
while (lo <= hi)
{
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    if (arr[lo] <= arr[mid])
    {
        if (arr[lo] <= target && target < arr[mid]) hi = mid - 1;
        else lo = mid + 1;
    }
    else
    {
        if (arr[mid] < target && target <= arr[hi]) lo = mid + 1;
        else hi = mid - 1;
    }
}

```

PRACTICE PROBLEMS

easy	Binary Search
easy	Search Insert Position
med	Search in Rotated Sorted Array
med	Find Minimum in Rotated Array
med	Koko Eating Bananas
med	Capacity to Ship Packages
hard	Median of Two Sorted Arrays

PITFALLS & TIPS

mid = (lo+hi)/2 overflows — always use lo + (hi-lo)/2.

hi = n vs n-1: lower-bound is open-right (n), classic is closed (n-1).

Binary search on answer: define feasibility predicate, find truth boundary.



Stack

Linear

T: $O(n)$

S: $O(n)$

WHEN TO USE

- Parentheses / matching
- Next greater / smaller
- Undo operations
- Expression evaluation

CODE TEMPLATES

Monotonic decreasing stack — next greater element

```
int[] result = new int[arr.Length];
Array.Fill(result, -1);
var stack = new Stack<int>(); // stores indices

for (int i = 0; i < arr.Length; i++)
{
    while (stack.Count > 0 && arr[stack.Peek()] < arr[i])
        result[stack.Pop()] = arr[i];
    stack.Push(i);
}
```

Valid parentheses

```
var stack = new Stack<char>();
var map = new Dictionary<char, char>
{ [')'] => '(', ['}]' => '{', ['}]' => '{' };

foreach (var c in s)
{
    if (!map.ContainsKey(c)) { stack.Push(c); continue; }
    if (stack.Count == 0 || stack.Pop() != map[c]) return false;
}
return stack.Count == 0;
```

Largest rectangle in histogram

```
var stack = new Stack<int>();
int max = 0;
int[] h = heights.Append(0).ToArray(); // sentinel

for (int i = 0; i < h.Length; i++)
{
    while (stack.Count > 0 && h[stack.Peek()] > h[i])
    {
        int height = h[stack.Pop()];
        int width = stack.Count == 0 ? i : i - stack.Peek() - 1;
        max = Math.Max(max, height * width);
    }
    stack.Push(i);
}
return max;
```

PRACTICE PROBLEMS

easy	Valid Parentheses
med	Min Stack
easy	Next Greater Element I
med	Daily Temperatures
hard	Largest Rectangle in Histogram
med	Decode String
med	Remove K Digits

PITFALLS & TIPS

Always guard with `Count > 0` before `Peek()` / `Pop()`.

Store indices, not values, when width matters (histogram).

Append sentinel 0 to histogram to flush the stack at end.



Queue / BFS

Graph

T: $O(V + E)$

S: $O(V)$

WHEN TO USE

- Shortest path (unweighted)
- Level-order traversal
- Grid / matrix traversal
- Multi-source BFS

CODE TEMPLATES

Graph BFS with visited set

```
var queue = new Queue<int>();
var visited = new HashSet<int>();
queue.Enqueue(start);
visited.Add(start);

while (queue.Count > 0)
{
    int node = queue.Dequeue();
    foreach (int nei in graph[node])
        if (visited.Add(nei)) // false = already visited
            queue.Enqueue(nei);
}
```

BFS level by level

```
var queue = new Queue<TreeNode>();
queue.Enqueue(root);

while (queue.Count > 0)
{
    int size = queue.Count; // snapshot current level
    for (int i = 0; i < size; i++)
    {
        var node = queue.Dequeue();
        if (node.left != null) queue.Enqueue(node.left);
        if (node.right != null) queue.Enqueue(node.right);
    }
}
```

Multi-source BFS (e.g. rotting oranges)

```

var queue = new Queue<(int r, int c)>();
for (int r = 0; r < rows; r++)
for (int c = 0; c < cols; c++)
    if (grid[r][c] == 2) queue.Enqueue((r, c));

int time = 0;
int[] dr = {-1,1,0,0}, dc = {0,0,-1,1};

while (queue.Count > 0)
{
    time++;
    int sz = queue.Count;
    for (int i = 0; i < sz; i++)
    {
        var (r, c) = queue.Dequeue();
        for (int d = 0; d < 4; d++)
        {
            int nr = r + dr[d], nc = c + dc[d];
            if (InBounds(nr, nc) && grid[nr][nc] == 1)
            {
                grid[nr][nc] = 2;
                queue.Enqueue((nr, nc));
            }
        }
    }
}

```

PRACTICE PROBLEMS

med	Binary Tree Level Order Traversal
med	Rotting Oranges
med	Number of Islands
hard	Word Ladder
med	01 Matrix
med	Shortest Path in Binary Matrix

PITFALLS & TIPS

Mark visited BEFORE enqueue or nodes queue multiple times → TLE.

Take level size snapshot BEFORE the inner for loop, not inside.

Multi-source: seed all sources at distance 0, BFS gives min from any.



DFS / Backtracking

Graph

T: $O(b^d)$
backtrack

S: $O(V)$ stack

WHEN TO USE

- All paths / combinations
- Undo-able choices
- Maze / grid reachability
- Tree recursion

CODE TEMPLATES

Backtracking (subsets / permutations)

```
var result = new List<IList<int>>();
var current = new List<int>();

void Backtrack(int start)
{
    result.Add(new List<int>(current)); // record state

    for (int i = start; i < nums.Length; i++)
    {
        current.Add(nums[i]);          // choose
        Backtrack(i + 1);              // explore
        current.RemoveAt(current.Count - 1); // un-choose
    }
}
Backtrack(0);
```

Grid DFS — flood fill / islands

```
void DFS(int[][] grid, int r, int c)
{
    if (r < 0 || r >= grid.Length ||
        c < 0 || c >= grid[0].Length ||
        grid[r][c] != 1) return;

    grid[r][c] = 0; // mark visited in-place
    DFS(grid, r+1, c); DFS(grid, r-1, c);
    DFS(grid, r, c+1); DFS(grid, r, c-1);
}
```

Iterative DFS (graph)

```
var stack = new Stack<int>();
var visited = new HashSet<int>();
stack.Push(start);

while (stack.Count > 0)
{
    int node = stack.Pop();
    if (!visited.Add(node)) continue;
    foreach (int nei in graph[node])
        if (!visited.Contains(nei)) stack.Push(nei);
}
```

PRACTICE PROBLEMS

med	Subsets
med	Permutations
med	Combination Sum
hard	N-Queens
med	Word Search
med	Pacific Atlantic Water Flow
med	Clone Graph

PITFALLS & TIPS

Forgetting un-choose step corrupts all future branches.

Deep recursion → stack overflow; convert to iterative.

Mutate grid in-place for visited (then restore) — saves $O(n)$ space.



Linked List

Linear

T: $O(n)$

S: $O(1)$

WHEN TO USE

- Reverse in-place
- Middle / k-th node
- Cycle detection
- Merge sorted lists

CODE TEMPLATES

Reverse linked list (iterative)

```
ListNode prev = null, curr = head;
while (curr != null)
{
    var next = curr.next; // save before redirect
    curr.next = prev;
    prev = curr;
    curr = next;
}
return prev;
```

Fast & slow pointer — middle / cycle

```
var slow = head;
var fast = head;

while (fast?.next != null)
{
    slow = slow.next;
    fast = fast.next.next;
}
// slow == middle
// if fast == slow (after loop) → cycle detected
// cycle start: reset slow = head, step one at a time
```

Remove N-th node from end

```
var dummy = new ListNode(0) { next = head };
var fast = dummy;
var slow = dummy;

for (int i = 0; i <= n; i++) fast = fast.next;

while (fast != null) { slow = slow.next; fast = fast.next; }
slow.next = slow.next.next;
return dummy.next;
```

PRACTICE PROBLEMS

easy	Reverse Linked List
easy	Detect Cycle
easy	Find Middle
easy	Merge Two Sorted Lists
med	Remove Nth Node From End
med	Reorder List
med	LRU Cache

PITFALLS & TIPS

Save next BEFORE redirecting curr.next or you lose the chain.

Use fast?.next != null (null-conditional) to avoid NPE.

Use a dummy head node to handle edge cases on head removal.



Tree / Recursion

Tree

T: $O(n)$

S: $O(h)$

WHEN TO USE

- Hierarchy traversal
- Height / depth queries
- LCA / path problems
- BST search / insert

CODE TEMPLATES

DFS traversals — pre / in / post

```
void PreOrder(TreeNode node)
{
    if (node == null) return;
    Process(node);          // pre-order
    PreOrder(node.left);
    PreOrder(node.right);
}
// InOrder: process BETWEEN children (BST sorted)
// PostOrder: process AFTER both children (height, diameter)
```

Return height up — diameter pattern

```
int diameter = 0;

int Height(TreeNode node)
{
    if (node == null) return 0;
    int left = Height(node.left);
    int right = Height(node.right);
    diameter = Math.Max(diameter, left + right); // global
    return 1 + Math.Max(left, right);
}
```

Lowest Common Ancestor (BST)

```
TreeNode LCA(TreeNode root, TreeNode p, TreeNode q)
{
    if (root == null) return null;
    if (p.val < root.val && q.val < root.val)
        return LCA(root.left, p, q);
    if (p.val > root.val && q.val > root.val)
        return LCA(root.right, p, q);
    return root; // split point is the answer
}
```

PRACTICE PROBLEMS

easy	Maximum Depth
easy	Invert Binary Tree
easy	Diameter of Binary Tree
easy	Balanced Binary Tree
med	Lowest Common Ancestor
hard	Binary Tree Max Path Sum
hard	Serialize / Deserialize Binary Tree

PITFALLS & TIPS

Computing height twice per node $\rightarrow O(n^2)$. Compute once and return up.

Null check must be the first thing in every recursive function.

Pass diameter as a ref param or closure — don't try to return it.



Heap / Priority Queue

Tree

T: $O(n \log k)$

S: $O(k)$

WHEN TO USE

- Top-K elements
- K-th largest / smallest
- Merge K sorted lists
- Median of stream

CODE TEMPLATES

Min-heap — PriorityQueue (.NET 6+)

```
var pq = new PriorityQueue<int, int>();

foreach (var x in arr)
    pq.Enqueue(x, x);    // (element, priority)

while (pq.Count > 0)
    int val = pq.Dequeue();    // removes min-priority
```

Top-K largest (min-heap of size k)

```
var pq = new PriorityQueue<int, int>();

foreach (var x in nums)
{
    pq.Enqueue(x, x);
    if (pq.Count > k) pq.Dequeue();    // evict smallest
}
// pq now holds the K largest elements
```

Two-heap median from data stream

```
var lo = new PriorityQueue<int,int>();    // max-heap (negate)
var hi = new PriorityQueue<int,int>();    // min-heap

void Add(int num)
{
    lo.Enqueue(num, -num);
    hi.Enqueue(lo.Peek(), lo.Dequeue());
    if (lo.Count < hi.Count)
    {
        int t = hi.Dequeue();
        lo.Enqueue(t, -t);
    }
}

double Median() =>
    lo.Count > hi.Count ? lo.Peek()
    : (lo.Peek() + hi.Peek()) / 2.0;
```

PRACTICE PROBLEMS

med Kth Largest Element in Array

med Top K Frequent Elements

hard Find Median from Data Stream

hard Merge K Sorted Lists

med Task Scheduler

med K Closest Points to Origin

PITFALLS & TIPS

C# PriorityQueue is min-heap only — negate priority for max-heap.

UnorderedItems.First() is NOT the top — use Peek().

Two-heap trick: $O(\log n)$ add, $O(1)$ median.



Dynamic Programming

DP

T: $O(n^2)$
typical

S: $O(n)$ or
 $O(n^2)$

WHEN TO USE

- Overlapping subproblems
- Optimal substructure
- Count / min / max ways
- Decision at each step

CODE TEMPLATES

1D DP — climbing stairs / Fibonacci

```
int[] dp = new int[n + 1];
dp[0] = 1; dp[1] = 1;

for (int i = 2; i <= n; i++)
    dp[i] = dp[i-1] + dp[i-2];

return dp[n];
```

2D DP — longest common subsequence

```
int[,] dp = new int[m+1, n+1];

for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++)
    {
        if (s[i-1] == t[j-1])
            dp[i,j] = dp[i-1,j-1] + 1;
        else
            dp[i,j] = Math.Max(dp[i-1,j], dp[i,j-1]);
    }

return dp[m, n];
```

0/1 Knapsack (iterate backwards)

```
int[] dp = new int[capacity + 1];

foreach (var (weight, value) in items)
    for (int c = capacity; c >= weight; c--) // backwards!
        dp[c] = Math.Max(dp[c], dp[c - weight] + value);

return dp[capacity];
```

Kadane's — max subarray

```
int maxSum = arr[0], curr = arr[0];

for (int i = 1; i < arr.Length; i++)
{
    curr = Math.Max(arr[i], curr + arr[i]);
    maxSum = Math.Max(maxSum, curr);
}
```

PRACTICE PROBLEMS

easy	Climbing Stairs
med	Maximum Subarray (Kadane)
med	House Robber
med	Longest Common Subsequence
med	0/1 Knapsack
med	Coin Change
hard	Edit Distance
med	Longest Increasing Subsequence

PITFALLS & TIPS

Iterating knapsack forward → items counted multiple times. Go backwards.

dp[0] is usually the empty-state base case, not first element.

Reusable items (unbounded knapsack)? Iterate forward. One-use? Backwards.

Space-optimize 2D DP: rolling two rows reduces $O(m \cdot n)$ to $O(n)$.



Graph Algorithms

Graph

T: $O((V+E) \log V)$

S: $O(V + E)$

WHEN TO USE

- Weighted shortest path
- Cycle detection
- Topological sort
- Union-Find / connectivity

CODE TEMPLATES

Dijkstra's shortest path

```
var dist = new int[n];
Array.Fill(dist, int.MaxValue);
dist[src] = 0;

var pq = new PriorityQueue<(int node, int d), int>();
pq.Enqueue((src, 0), 0);

while (pq.Count > 0)
{
    var (node, d) = pq.Dequeue();
    if (d > dist[node]) continue; // stale entry

    foreach (var (nei, w) in graph[node])
    {
        int nd = dist[node] + w;
        if (nd < dist[nei])
        {
            dist[nei] = nd;
            pq.Enqueue((nei, nd), nd);
        }
    }
}
```

Topological sort — Kahn's (BFS)

```
int[] inDegree = new int[n];
foreach (var (u, v) in edges) inDegree[v]++;

var queue = new Queue<int>(
    Enumerable.Range(0, n).Where(i => inDegree[i] == 0));
var order = new List<int>();

while (queue.Count > 0)
{
    int node = queue.Dequeue();
    order.Add(node);
    foreach (int nei in graph[node])
        if (--inDegree[nei] == 0) queue.Enqueue(nei);
}
// order.Count < n → cycle exists
```

Union-Find (path compression + rank)

```

int[] parent = Enumerable.Range(0, n).ToArray();
int[] rank   = new int[n];

int Find(int x) =>
    parent[x] == x ? x : parent[x] = Find(parent[x]);

bool Union(int x, int y)
{
    int px = Find(x), py = Find(y);
    if (px == py) return false;    // already connected
    if (rank[px] < rank[py]) (px, py) = (py, px);
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

```

PRACTICE PROBLEMS

med	Network Delay Time (Dijkstra)
med	Course Schedule (topo sort)
med	Number of Connected Components
med	Cheapest Flights Within K Stops
med	Redundant Connection (Union-Find)
hard	Critical Connections in a Network

PITFALLS & TIPS

Dijkstra fails on negative weights — use Bellman-Ford instead.

Skip stale PQ entries: if ($d > \text{dist}[\text{node}]$) continue.

Union-Find with path compression + rank is near $O(1)$ per op.